



ΔΗΜΟΚΡΙΤΕΙΟ
ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΡΑΚΗΣ

ΤΜΗΜΑ
ΗΜ&ΜΥ

Αναγνώριση Προτύπων

2023-2024

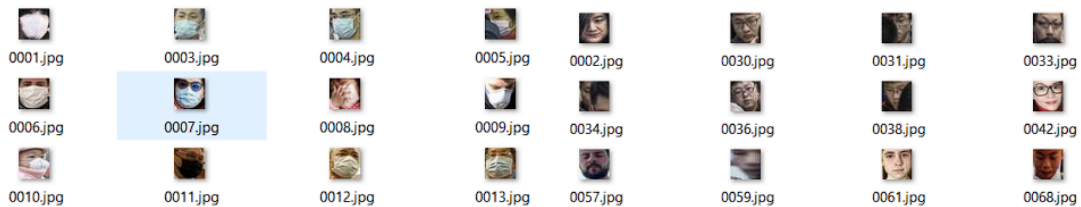
Final Project

Ονοματεπώνυμο: Δημήτριος Αναστασούδης
ΑΕΜ: 58085

Άσκηση 1.

1. Μέρος 1ο – Ταξινόμηση Εικόνων (50%)

Στο πρώτο μέρος καλείστε να επιλύσετε ένα πρόβλημα ταξινόμησης εικόνων που προκύπτουν από αλγορίθμους ανίχνευσης προσώπων, με στόχο την επιβεβαίωση χρήσης μάσκας. Τα δεδομένα που θα χρησιμοποιήσετε βρίσκονται στο αρχείο Mask_DB.zip, και αποτελούνται από 1044 εικόνες προσώπων χωρίς τη χρήση μάσκας (without_mask), ισάριθμες εικόνες προσώπων που χρησιμοποιούν μάσκα (with_mask), και 56 εικόνες προσώπων που χρησιμοποιούν μάσκα εσφαλμένα (mask_incorrect_use).



Καταρχάς, φορτώνουμε το Dataset με τον εξής τρόπο:

```
[3] import zipfile
import os
import cv2
import numpy as np

zip_path = '/content/Mask_DB.zip'

# Αποσυμπίεση του αρχείου ZIP
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('/content')

# Φόρτωση των εικόνων
def load_images_from_folder(folder):
    images = []
    for filename in os.listdir(folder):
        img = cv2.imread(os.path.join(folder, filename))
        if img is not None:
            images.append(img)
    return images

# Φόρτωση εικόνων από κάθε κατηγορία
without_mask = load_images_from_folder('/content/without_mask')
with_mask = load_images_from_folder('/content/with_mask')
mask_incorrect = load_images_from_folder('/content/mask_incorrect_use')

# Ετικέτες (0 = without_mask, 1 = with_mask, 2 = mask_incorrect)
labels_all = [0] * len(without_mask) + [1] * len(with_mask) + [2] * len(mask_incorrect)

# Συνδυασμός όλων των εικόνων
all_images = without_mask + with_mask + mask_incorrect

# Μετατροπή σε numpy arrays
all_images_np = np.array(all_images)
all_labels_np = np.array(labels_all)
```

Καθώς ο κώδικας τρέχει στο Google Colab, είναι σημαντικό να σιγουρευτούμε ότι έχουμε ανεβάσει το αρχείο Mask_DB.zip μέσα στον φάκελο Content του εργαλείου αυτού. Επίσης,

πρέπει να φροντίσουμε ότι αυτό όντως βρίσκεται εκεί, και αν δεν βρίσκεται να το ξαναανεβάσουμε, καθώς όταν τελειώσει το runtime μας, τα αρχεία μας διαγράφονται από τους φακέλους.

Έτσι, αν τα έχουμε όλα σωστά, παίρνουμε τα παρακάτω αποτελέσματα:

```
Αριθμός εικόνων χωρίς μάσκα: 1044
Αριθμός εικόνων με μάσκα: 1044
Αριθμός εικόνων με λάθος χρήση μάσκας: 56
Αριθμός όλων των εικόνων: 2144
Αριθμός όλων των labels: 2144
```

a)

- a. Ως δεδομένα εκπαίδευσης, επικύρωσης και δοκιμής θα χρησιμοποιηθούν **μόνο** εικόνες από τις κλάσεις `with_mask` και `without_mask` με τις αντίστοιχες ετικέτες. Χωρίστε τα δεδομένα τυχαία σε 60% δεδομένα εκπαίδευσης, 20% δεδομένα επικύρωσης και 20% δεδομένα δοκιμής.

Αυτό, επιτυγχάνεται με τον παρακάτω κώδικα:

```
[4]: from sklearn.model_selection import train_test_split

images_a = with_mask + without_mask
labels_a = [0] * len(without_mask) + [1] * len(with_mask)

# Μετατροπή σε numpy arrays
images_np = np.array(images_a)
labels_np = np.array(labels_a)

# Διαχωρισμός σε σύνολα εκπαίδευσης, επικύρωσης και δοκιμής
train_images, test_val_images, train_labels, test_val_labels = train_test_split(images_np, labels_np, test_size=0.4, random_state=42)
val_images, test_images, val_labels, test_labels = train_test_split(test_val_images, test_val_labels, test_size=0.5, random_state=42)

total_images = len(images_a)
print(f"Ποσοστό εικόνων εκπαίδευσης: {len(train_images) / total_images * 100:.2f}%")
print(f"Ποσοστό εικόνων επικύρωσης: {len(val_images) / total_images * 100:.2f}%")
print(f"Ποσοστό εικόνων δοκιμής: {len(test_images) / total_images * 100:.2f}%")

Ποσοστό εικόνων εκπαίδευσης: 59.96%
Ποσοστό εικόνων επικύρωσης: 20.02%
Ποσοστό εικόνων δοκιμής: 20.02%
```

Παρατηρούμε ότι τα ποσοστά, δεν είναι ακριβώς αυτό που το ζητήσαμε, δηλαδή 60%. Αυτό, συμβαίνει, καθώς ο συνολικός αριθμός των δεδομένων δεν διαιρείται ακριβώς.

b)

- b. Υλοποιείστε και εκπαιδέυστε ένα σύστημα αναγνώρισης προτύπων χρησιμοποιώντας κατάλληλους αλγορίθμους της επιλογής σας συμβατικούς¹ ή/και νευρωνικούς. Δικαιολογήστε το σκεπτικό σας.

Οι επιλογές που είχα σε αυτή την υλοποίηση ήταν πολλές. Από συμβατικούς αλγόριθμους, οι σκέψεις μου ήταν να χρησιμοποιήσω ένα SVM (Support Vector Machine) ή ένα Random Forest. Ωστόσο, επέλεξα να πραγματοποιήσω την υλοποίηση με ένα Νευρωνικό δίκτυο, και πιο συγκεκριμένα ένα CNN (Συνελικτικό Νευρωνικό Δίκτυο).

Αυτή η επιλογή έγινε καθώς η εργασία αφορά την ανάλυση πολλών εικόνων από το Dataset μας. Σε αυτές τις περιπτώσεις, ένα CNN είναι αρκετά καλό, καθώς είναι σχεδιασμένο να αναγνωρίζει τα pattern σε εικόνες μέσω της εκμάθησης χαρακτηριστικών. Αυτό, το κάνει πιο αποδοτικά με μεγαλύτερο accuracy από τις υπόλοιπες τεχνικές. Βέβαια, απαιτεί και περισσότερα resources και χρόνο εκπαίδευσης, κάτι το οποίο παρατηρήθηκε, αλλά δεν ήταν σε απαγορευτικό σημείο. Τέλος, επίσης υπήρχε και ένα κίνδυνος overfitting, το οποίο δεν θέλουμε και το λήφθηκε κατά νου.

Ο κώδικας, βρίσκεται στο αρχείο "AP_Final_1_58085.ipynb" και μας δίνει το παρακάτω output:

```
Epoch 10/10  
40/40 ----- 10s 136ms/step - accuracy: 0.9761 - loss: 0.0773 - val_accuracy: 0.9809 - val_loss: 0.0771
```

Δηλαδή, καλούς αριθμούς στο accuracy και val_accuracy, με μικρά loss.

c)

- c. Καθορίστε τις βέλτιστες παραμέτρους του μοντέλου σας (πχ. αριθμό ελάττωσης διαστάσεων, παραμέτρους ταξινομητών, μοντέλα δικτύων κλπ) χρησιμοποιώντας το σύνολο επικύρωσης. Δώστε μερικά παραδείγματα παραλλαγών που δοκιμάσατε και την επίδοσή τους στο validation set. Για το καλύτερο μοντέλο σας υπολογίστε το σφάλμα ταξινόμησης στο σύνολο δοκιμής, καθώς και τις καμπύλες precision-recall και το AUC.

Για τον καθορισμό των βέλτιστων παραμέτρων, υλοποιήθηκε ένας αυτοματισμός με την χρήση του keras tuner. Με τον tuner αυτόν, πραγματοποιούνται trials κατά τα οποία δοκιμάζονται οι υπερπαραμέτροι του CNN μας, μέχρι να εντοπιστεί ο καλύτερος συνδυασμός σύμφωνα με τους περιορισμούς που του αναθέσαμε. Αυτός ο tuner, αρχικά δοκιμάζει συνδυασμούς με λιγότερα epochs, μέχρι να εντοπίσει τους καλύτερους. Εφόσον γίνει αυτό, "πετάει" τους άλλους, και συνεχίζει με τους καλύτερους με ολοένα και περισσότερα epochs μέχρι να βρεθεί ο καλύτερος συνδυασμός υπερπαραμέτρων. Επιπλέον, υπάρχει και μία λειτουργία stop early, η οποία έχει έναν αριθμό patience (patience=3), κατά την οποία αν δεν δει βελτίωση του accuracy μέσα σε 3 εποχές, παρατάει τον συνδυασμό και προχωράει στον επόμενο. Καθώς οι max epochs που ορίστηκαν ήταν 10, επιλέχθηκε ο αριθμός patience=3, ωστόσο με μεγαλύτερο αριθμό patience, είχαμε λίγο καλύτερα αποτελέσματα.

Μερικά παραδείγματα καλών παραμέτρων:

```
Trial 0025 summary
Hyperparameters:
conv_1_filter: 64
conv_2_filter: 32
conv_3_filter: 64
dense_1_units: 96
tuner/epochs: 10
tuner/initial_epoch: 4
tuner/bracket: 1
tuner/round: 1
tuner/trial_id: 0018
Score: 0.980861246585846
```

```
Trial 0024 summary
Hyperparameters:
conv_1_filter: 128
conv_2_filter: 64
conv_3_filter: 64
dense_1_units: 32
tuner/epochs: 10
tuner/initial_epoch: 4
tuner/bracket: 1
tuner/round: 1
tuner/trial_id: 0022
Score: 0.9832535982131958
```

```
Trial 0008 summary
Hyperparameters:
conv_1_filter: 32
conv_2_filter: 48
conv_3_filter: 64
dense_1_units: 80
tuner/epochs: 2
tuner/initial_epoch: 0
tuner/bracket: 2
tuner/round: 0
Score: 0.9784688949584961
```

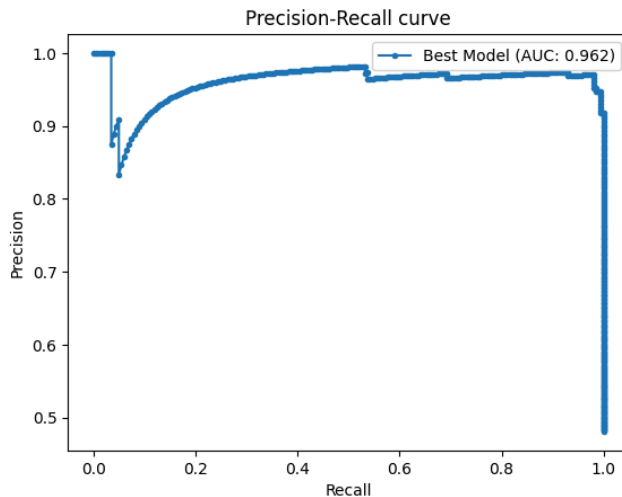
Οι βέλτιστες υπερπαράμετροι φαίνονται παρακάτω:

```
Βέλτιστες υπερπαράμετροι:
conv_1_filter: 48
conv_2_filter: 64
conv_3_filter: 128
dense_1_units: 32
```

Ανάλογα με τις τιμές σε παραμέτρους που έθετα, όπως αυτού για την λειτουργία stop early, των εποχών, αλλά και του batch size, έπαιρνα διαφορετικά αποτελέσματα και υπήρχε ποικιλία στο τελικό Accuracy.

Η τελική απώλεια και accuracy καθώς και η AUC φαίνεται παρακάτω:

Τελική απώλεια στο σύνολο δοκιμής (Test Loss): 0.1188
Τελική ακρίβεια στο σύνολο δοκιμής (Test Accuracy): 0.9713



d)

- d. Εφαρμόστε το εκπαιδευμένο μοντέλο σας στις εικόνες της κλάσης «mask_incorrect_use» και υπολογίστε το ποσοστό αυτών που ταξινομούνται εσφαλμένα ως αποδεκτή χρήση μάσκας. Προτείνετε τρόπους που μπορεί να αυξηθεί ο αριθμός των εικόνων της κλάσης mask_incorrect_use που ταξινομούνται ως μη χρήση μάσκας, χωρίς να επηρεαστούν σημαντικά τα αποτελέσματα του test set, χρησιμοποιώντας μόνο τα υπάρχοντα δεδομένα. Δοκιμάστε κάτι από αυτά που προτείνετε και δείξτε εάν έχει το επιθυμητό αποτέλεσμα.

Με το τρέξιμο του κώδικα, παίρνουμε το παρακάτω output:

Ποσοστό εικόνων 'mask_incorrect_use' που ταξινομήθηκαν εσφαλμένα ως 'with_mask': 14.29%

Τρόποι βελτίωσης του παραπάνω είναι οι εξής:

- Data Augmentation (προσθήκη εικόνων mask_incorrect_use) (δεν γίνεται στην περίπτωση μας)
- Εισαγωγή συναρτήσεων κόστους στις κλάσεις (δεν γίνεται στην περίπτωση μας)
 - Αυτή, αν και ήταν η πρώτη μου σκέψη, δεν υλοποιείται εύκολα στην περίπτωση μας. Αυτό, συμβαίνει καθώς η εκπαίδευση γίνεται μόνο για τις άλλες δύο περιπτώσεις.
- Χρήση transfer learning από μοντέλα που έχουν ήδη εκπαιδευτεί πάνω σε αυτό

- Το θεώρησα πιο περίπλοκο από κάποιους άλλους τρόπους οπότε δεν το ακολούθησα
- Anomaly Detection
 - Το anomaly detection είναι και η τεχνική που επέλεξα. Ουσιαστικά σύμφωνα με ένα threshold(score) που έβαλα στον κώδικα, κάνω το μοντέλο μου να αναγνωρίζει τις κλάσεις που διαφέρουν από τις κλάσεις without_mask και with_mask

Αν και στην αρχή μου ήταν δύσκολο, το τελικό αποτέλεσμα που πήρα ήταν αυτό:

```
Ποσοστό των εικόνων 'mask_incorrect_use' που ταξινομήθηκαν ως anomalies: 91.07%
Ποσοστό των εικόνων 'mask_incorrect_use' που ταξινομήθηκαν εσφαλμένα ως 'with_mask': 8.93%
```

Αυτό το αποτέλεσμα επιτεύχθηκε με ένα πολύ μικρό threshold-score, με τιμή 0.0005. Αυτό, ίσως επιφέρει διάφορα προβλήματα όπως η αύξηση των False Positives, η μείωση της ευαισθησίας και της απόδοσης.

Άσκηση 2.

2. Μέρος 2ο – Πρόβλεψη πρόσδεσης χημικών μορίων σε βιολογικούς υποδοχείς (50%)

Στο δεύτερο μέρος καλείστε να επιλύσετε ένα πρόβλημα πρόβλεψης του βαθμού πρόσδεσης (binding) χημικών μορίων σε βιολογικούς υποδοχείς. Η διαδικασία αυτή χρησιμοποιείται συχνά στη φαρμακολογία και τη βιοχημεία για την ανακάλυψη χημικών μορίων που δυνητικά μπορούν να αποτελέσουν ενεργές ουσίες που προσδένονται σε κάποιο υποδοχέα-στόχο και αναστέλλουν τη δράση του. Στο αρχείο Data_Receptors.zip θα βρείτε δεδομένα εκπαίδευσης και δοκιμής για 1115 και 124 χημικά μόρια αντίστοιχα, για καθένα από τα οποία έχουν υπολογισθεί 3473 χαρακτηριστικά. Τα πρώτα 1425 χαρακτηριστικά (κατά στήλες) μπορούν να λάβουν συνεχείς τιμές, ενώ τα 2048 επόμενα αποτελούν binary περιγραφείς των μορίων. Για τα δεδομένα εκπαίδευσης σας δίνονται και οι ετικέτες που καθορίζουν εάν το κάθε μόριο μπορεί να προσδεθεί αποτελεσματικά στον υποδοχέα στόχο (label 1) ή όχι (label 0).

Καταρχάς, φορτώνουμε το Dataset με τον εξής τρόπο:

```
import pandas as pd
import zipfile
import os

zip_path = '/content/Data_Receptors.zip'

# Αποσυμπίεση του zip αρχείου και φόρτωση των δεδομένων
with zipfile.ZipFile(zip_path, 'r') as zip_ref:

    zip_ref.extractall('/content')

# Φόρτωση των δεδομένων εκπαίδευσης και τεστ
train_features = pd.read_csv('/content/Train_Features.csv')
train_labels = pd.read_csv('/content/Train_Labels.csv')
test_features = pd.read_csv('/content/Test_Features.csv')

# Εμφάνιση των πρώτων γραμμών από κάθε DataFrame για επιβεβαίωση
print("Train Features:")
print(train_features.head())
print("\nTrain Labels:")
print(train_labels.head())
print("\nTest Features:")
print(test_features.head())
```

Καθώς ο κώδικας τρέχει στο Google Colab, είναι σημαντικό να σιγουρευτούμε ότι έχουμε ανεβάσει το αρχείο Data_Receptors.zip μέσα στον φάκελο Content του εργαλείου αυτού. Επίσης, πρέπει να φροντίσουμε ότι αυτό όντως βρίσκεται εκεί, και αν δεν βρίσκεται να το ξαναανεβάσουμε, καθώς όταν τελειώσει το runtime μας, τα αρχεία μας διαγράφονται από τους φακέλους.

a) και b)

- a. Σχεδιάστε και υλοποιήστε ένα μοντέλο μηχανικής μάθησης χρησιμοποιώντας κατάλληλους αλγορίθμους της επιλογής σας, συμβατικούς ή/και νευρωνικούς, που δέχεται σαν είσοδο τα χαρακτηριστικά νέων χημικών μορίων και προβλέπει αν το κάθε μόριο μπορεί να συνδεθεί στον υποδοχέα στόχο. Δικαιολογήστε το σκεπτικό σας.
- b. Τα διαθέσιμα δεδομένα για βελτιστοποίηση και εκπαίδευση του μοντέλου σας είναι μονάχα αυτά που περιλαμβάνονται στο αρχείο Train_features.csv για τα οποία έχετε διαθέσιμες ετικέτες (Train_labels.csv). Ακολουθήστε το κατάλληλο πρωτόκολλο ώστε να βελτιστοποιήσετε τις παραμέτρους του

μοντέλου σας, και να εκτιμήσετε με τον καλύτερο δυνατό τρόπο το πιθανό σφάλμα στο test set.

Αρχικά, επιτεύχθηκε διαχωρισμός όσον αφορά το preprocessing των Data, ανάλογα με την κατηγορία τους, και επιτεύχθηκε το ανάλογο scaling εκεί που χρειάστηκε.

Το μοντέλο μηχανικής μάθησης μου, επέλεξα να το υλοποιήσω με την χρήση του συνδυασμό της PCA(ανάλυση κύριων συνιστωσών) και ενός CNN (Συνελικτικό Νευρωνικό Δίκτυο).

Αυτή η επιλογή έγινε με την σκέψη ότι η PCA θα μας μειώσει τις διαστάσεις των δεδομένων στο preprocessing, δηλαδή πριν την εκπαίδευση και το CNN θα μας δώσει καλές αποδόσεις στο accuracy και ένα σωστό αποτέλεσμα μέσω της αποτελεσματικής αναγνώρισης των προτύπων. Ο συνδυασμός αυτών των δύο θα υλοποιήσουν το επιθυμητό αποτέλεσμα, με έναν διαχειρίσιμο τρόπο (μετά την υλοποίηση της PCA).

Αν και σίγουρα, αυτός ο συνδυασμός απαιτεί υπολογιστικά resources και χρόνο εκπαίδευσης, τον προτίμησα από ένα SVM ή από άλλους συμβατικούς αλγόριθμους, καθώς αναμένω να παρέχει καλύτερα αποτελέσματα.

Για τον καθορισμό των βέλτιστων παραμέτρων, υλοποιήθηκε ένας αυτοματισμός με την χρήση του keras tuner. Με τον tuner αυτόν, πραγματοποιούνται trials κατά τα οποία δοκιμάζονται οι υπερπαραμέτροι του CNN μας, μέχρι να εντοπιστεί ο καλύτερος συνδυασμός σύμφωνα με τους περιορισμούς που του αναθέσαμε. Αυτός ο tuner, δοκιμάζει τις υπερπαραμέτρους που έχουν να κάνουν με τον αριθμό φίλτρων των επιπέδων, τα επίπεδα του CNN και την μεταβλητή dropout η οποία συμβάλλει στο να μην επιτευχθεί overfitting.

Το καλύτερο δυνατό accuracy που λάβαμε είναι το παρακάτω:

```
Accuracy: 90.58296084403992%
```

Οι καλύτερες παράμετροι επίσης είναι οι εξής:

```
Οι βέλτιστες υπερπαραμέτροι:  
input_units: 32  
n_layers: 3  
conv_0_units: 128  
dense_units: 96  
dropout: 0.30000000000000004  
conv_1_units: 64  
conv_2_units: 96
```

c)

- c. Δώστε την εκτίμησή σας για το σφάλμα που θα έχει το εκπαιδευμένο μοντέλο σας στο test set, δεδομένου ότι η κατανομή κλάσεων στο test set είναι παρόμοια με αυτή των δεδομένων εκπαίδευσης. Να εκτιμήσετε επίσης το AUC για την καμπύλη TP-FP. Τέλος, να παράξετε ένα αρχείο με όνομα *test_predictions.csv* στο οποίο θα καταχωρήσετε την πρόβλεψη του μοντέλου σας για κάθε δείγμα από το test set στη μορφή *predicted_label,prediction_score* σε κάθε γραμμή, με τη σειρά που περιέχονται τα δεδομένα στο *test_features.csv*. Το *prediction_score* αποτελεί την πρωτογενή έξοδο του ταξινομητή σας και πρέπει να είναι είτε η πιθανότητα πρόσδεσης είτε να έχει αύξουσα τιμή για αυξανόμενη πιθανότητα πρόσδεσης. Το αρχείο να συμπεριληφθεί μαζί με τα υπόλοιπα παραδοτέα της εργασίας στην υποβολή σας.

Το output που παίρνουμε από τον κώδικά μας φαίνεται παρακάτω:

```
7/7 [=====] - 0s 5ms/step - loss: 0.3258 - accuracy: 0.9058
Accuracy: 90.58296084403992%
7/7 [=====] - 0s 5ms/step
AUC: 0.9509803921568627
4/4 [=====] - 0s 9ms/step
```

Το accuracy είναι αυτό που υπολογίστηκε και προηγουμένως, και το AUC(Area Under Curve) έχει τη τιμή που φαίνεται παραπάνω, δηλαδή 0.9509 ή περίπου 95%.

Το αρχείο *test_predictions.csv* βρίσκεται μαζί με τα υπόλοιπα αρχεία στο φάκελο zip.